Introducere in Machine Learning

1. Structura proiectului:

Tema1-CA/
– notebooks/

– 01_eda.ipynb # Analiza datelor
– 02_preprocessing.ipynb # Preprocesarea datelor
– 03_classification.ipynb # Modele de clasificare
– 04_regression.ipynb # Modele de regresie
– eda_plots/ # Graficele generate in etapa de analiza
– preprocessed_data/ # Date preprocesate
– predictions_clf.csv # Predictii clasificare
– predictions_reg.csv # Predictii regresie
– CA_OUALD_train.csv # Set de antrenare
– CA_OUALD_test.csv # Set de validare
– CA_private_test.csv # Set de test Kaggle

*Pentru afisarea rezultatelor, notebook-urile trebuie rulate secvential, dupa numarul lor: eda -> preprocessing -> classification -> regression.

2. Explorarea datelor (EDA)

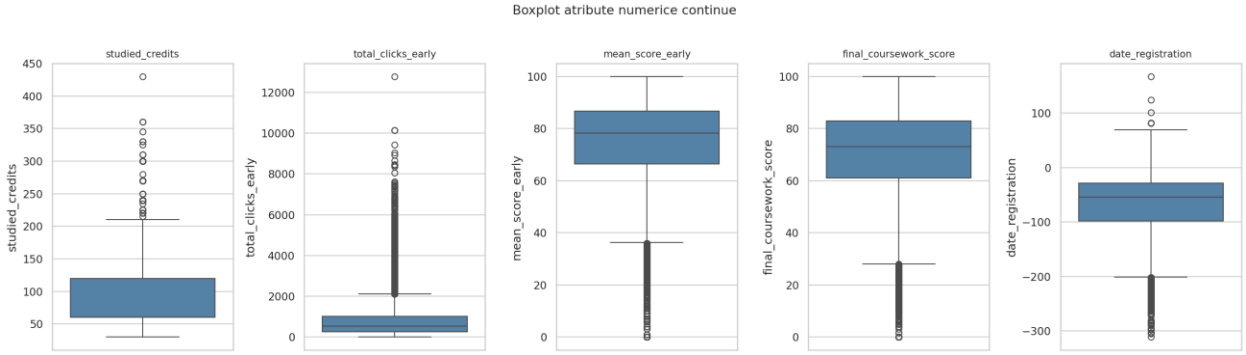
2.1. Analiza tipului de atribute:

Am clasificat atributele in 3 categorii distincte, dupa indicatii:

- Numerice (33): num_of_prev_attempts, studied_credits, date_registration, date_unregistration, module_presentation_length etc.
- Nominale (5): code_module, code_presentation, gender, region, disability
- Ordinale (4): highest_education, imd_band, age_band, clicks_freq_init

Pentru cele numerice am afisat tabelul cu statistici din care se poate observa cate coloane sunt lipsa pentru fiecare atribut si distributia valorilor pentru creearea unei prime idei despre date (din cauza ca sunt multe atribute numerice nu am inclus tabelul aici, dar se poate vedea in notebook).

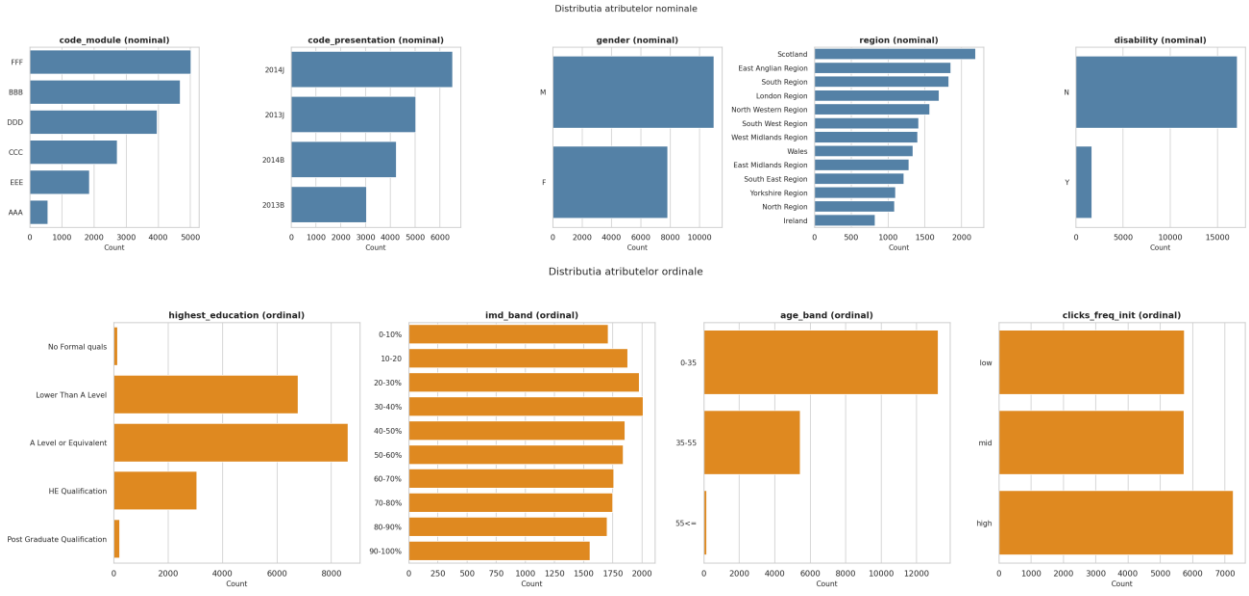
Am selectat cateva atribute numerice pentru care sa realizez boxplot-uri cu distributia valorilor:



- **studied_credits** are o distributie destul de concentrata, cu doar cativa outlieri in partea superioara care reprezinta studentii mult mai activi decat media.
- **total_clicks_early** prezinta outlieri semnificativi in partea superioara, ceea ce indica ca unii studenti sunt mult mai activi decat majoritatea.
- **mean_score_early** are mai multi outlieri catre 0 care semnifica studentii care nu au completat evaluarile
- **final_coursework_score** (obiectivul de regresie) are multe valori extreme
- **date_registration** arata ca majoritatea s-au inscris la cursuri inainte de inceperea acestora si foarte putine exceptii

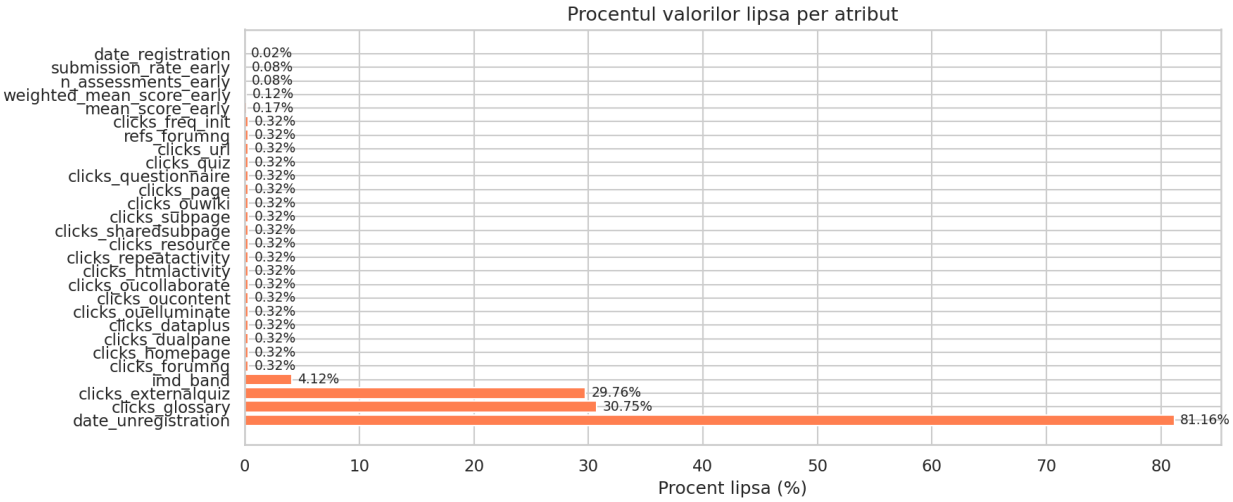
Pentru cele nominale si ordinale am facut la fel doua tabele cu datele cerute care ne arata ca nu exista coloane cu atribute nominale lipsa si cam la cate clase ar trebui sa ne asteptam pentru fiecare atribut.

Pentru toate atributele nominale si ordinale am realizat histograme pentru observarea mai usoara a detaliilor.

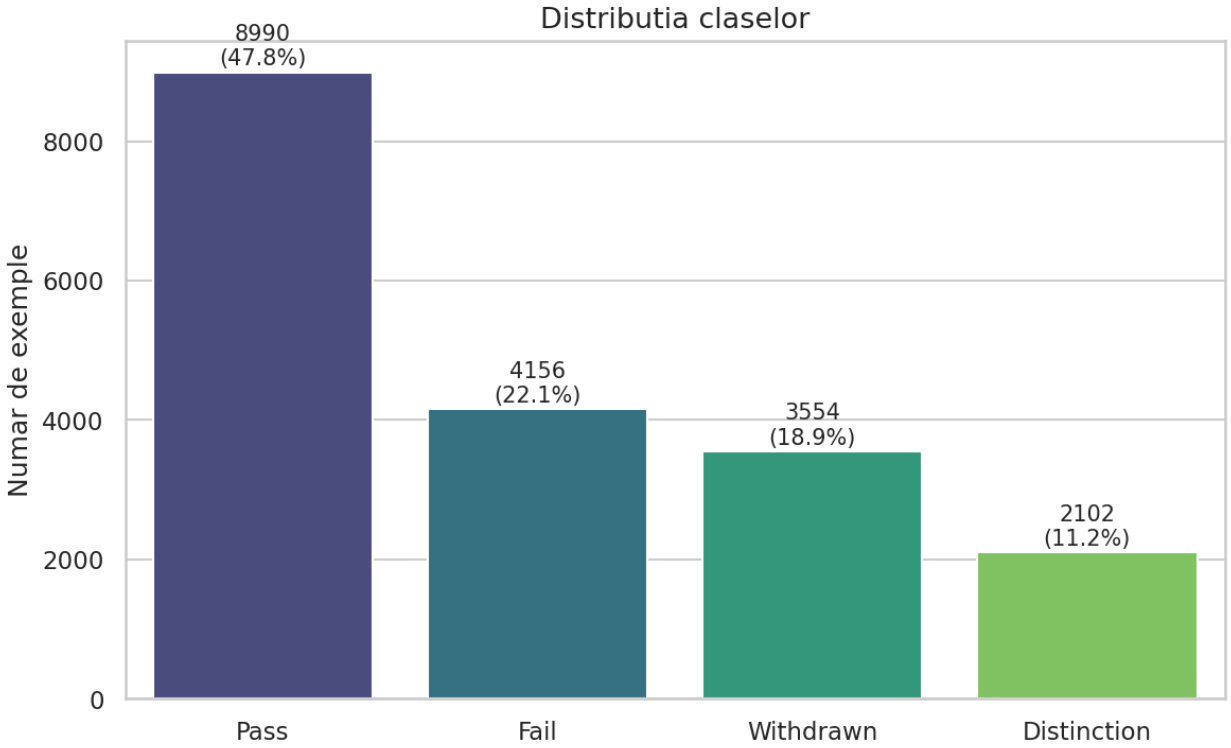


- **gender** are distributie relativ apropiata intre barbati si femei
- **region** ne arata ca majoritatea studentilor provin din zona de sud-est UK
- **disability** prezinta putini studenti cu dizabilitati
- **highest_education** este dominat de A level or Equivalent, dar cu destul de multi studenti Lower than A level
- **imd_band** arata calitatea internetului intre studenti fiind aproape egala distribuita intre clase
- **age_band** arata ca majoritatea studentilor sunt tineri sub 35 ani
- **clicks_freq_init** are o distributie pe 3 nivele cu high predominant

Bonus am mai adaugat acest grafic foarte ajutator pentru observarea numarului de valori lipsa per atribut:



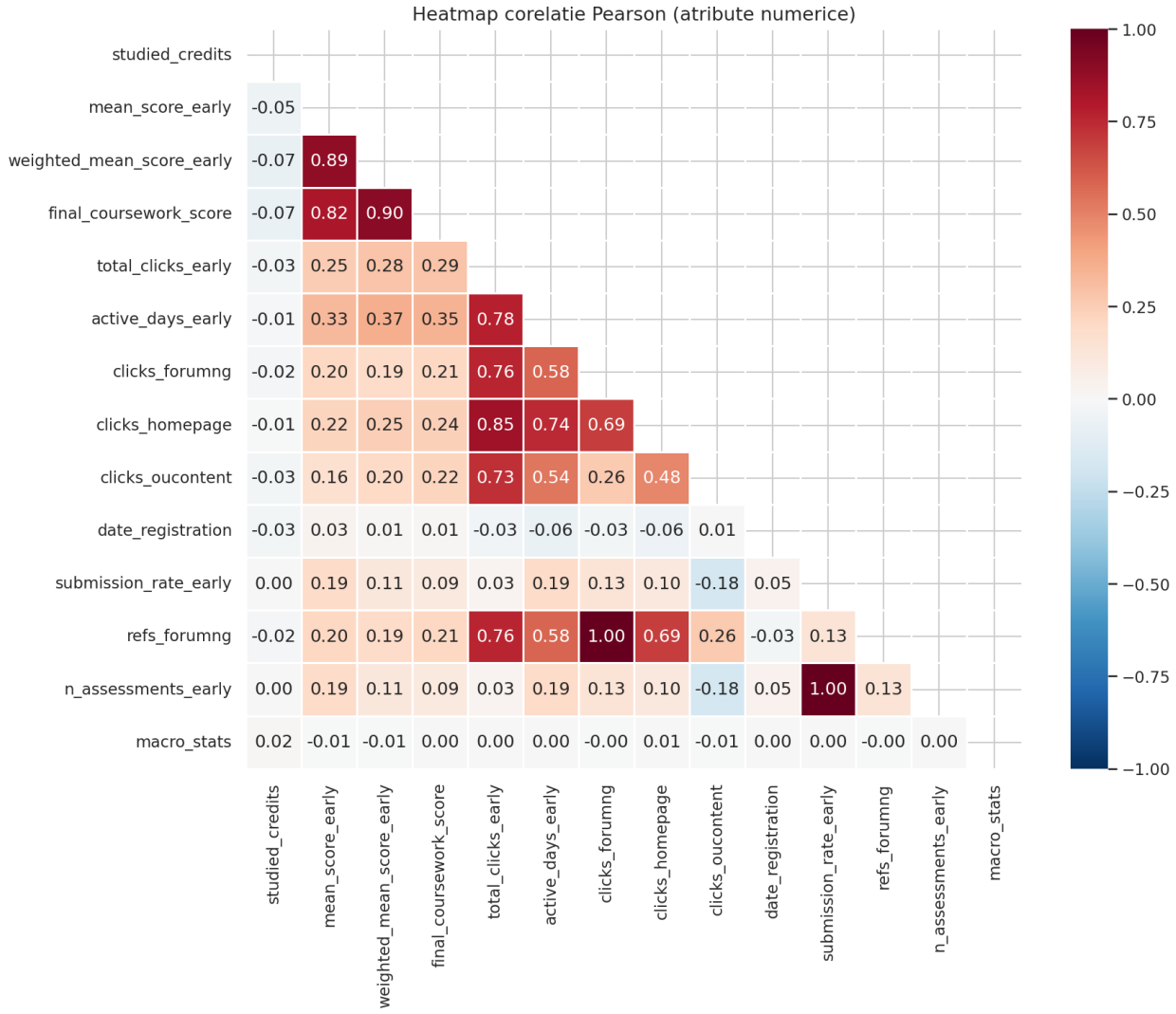
2.2. Analiza echilibrului de clase



Distributia obiectivului **final_result** este dezechilibrata. Clasa Distinction are o pondere foarte mica, Withdrawn si Fail au diferenta ceva mai mica, insa Pass domina cu o pondere de peste doua ori mai mare decat Fail. Acest lucru face ca acuratetea sa nu fie o metrica precisa, deoarece un model naiv ar putea prezice doar clasa Pass si sa aiba o

acuratete de ~48%. Astfel, ne vom raporta la F1 care este preferata in special pentru clase dezechilibrate, deoarece nu este influentata de valori extreme.

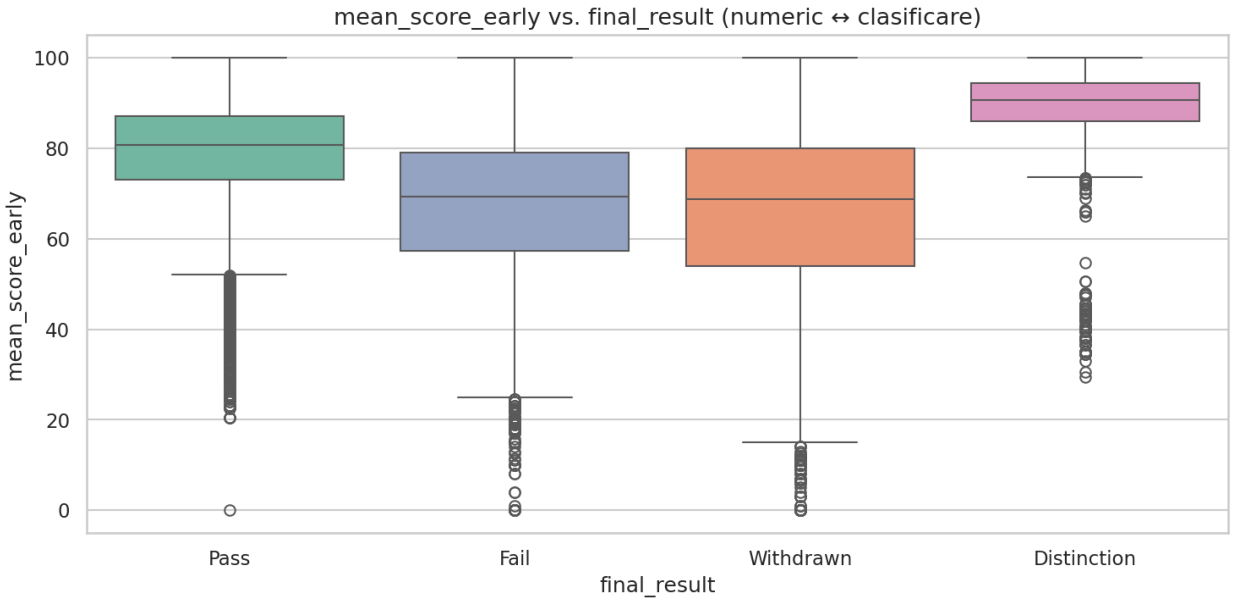
2.3. Analiza corelatiei intre atribute



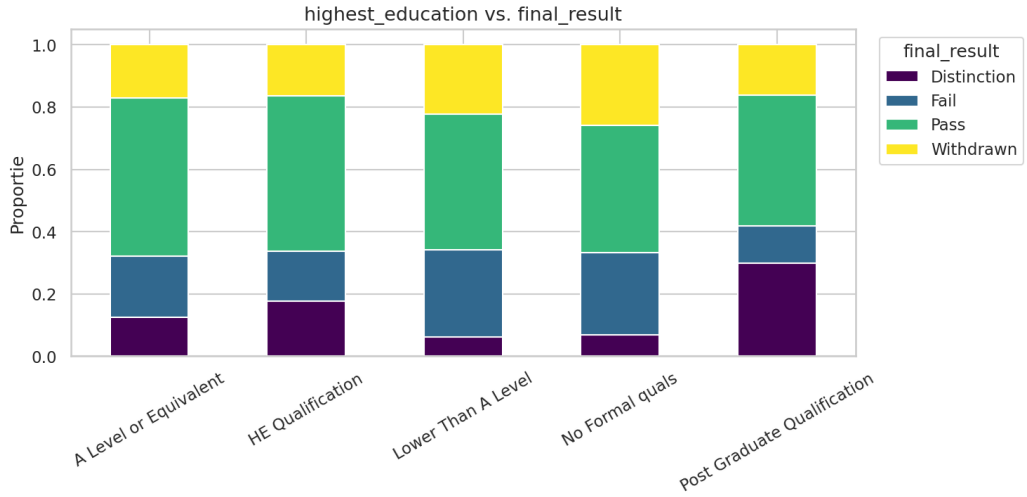
Heatmap-ul de corelatie Pearson identifica:

- **mean_score_early** si **weighted_mean_score_early** sunt puternic corelate (0.97) - redundante
- **clicks_forumng** si **refs_forumng** sunt puternic corelate (0.99) - redundante
- **total_clicks_early** si **active_days_early** sunt destul de corelate (0.78)
- **Mean_score_early** si **final_coursework_score** sunt destul de corelate (0.8)

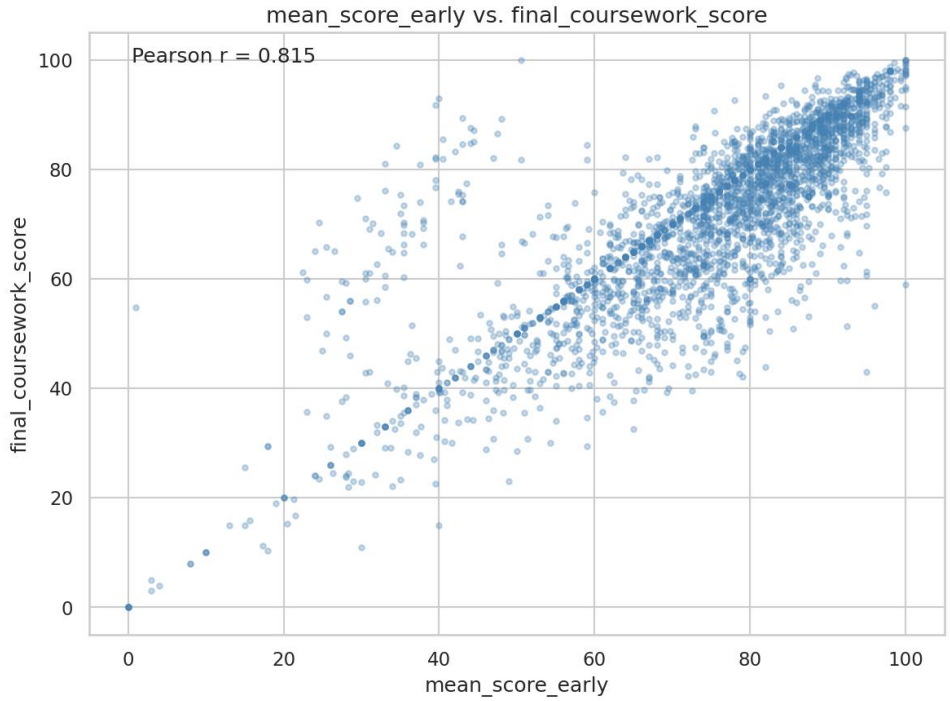
Atribut numeric -> Clasificare: **mean_score_early** vs **final_result** a fost analizata folosind metoda Kruskal-Wallis care determina daca exista diferente semnificative intre 2 sau mai mule grupuri atunci cand nu exista o distributie normala a datelor. Rezultatul ne arata ca distributiile mean_score_early difera semnificativ intre clase: studentii cu Distinction au mediane mai ridicate, iar cei cu Withdrawn/Fail au mediane mult mai joase.



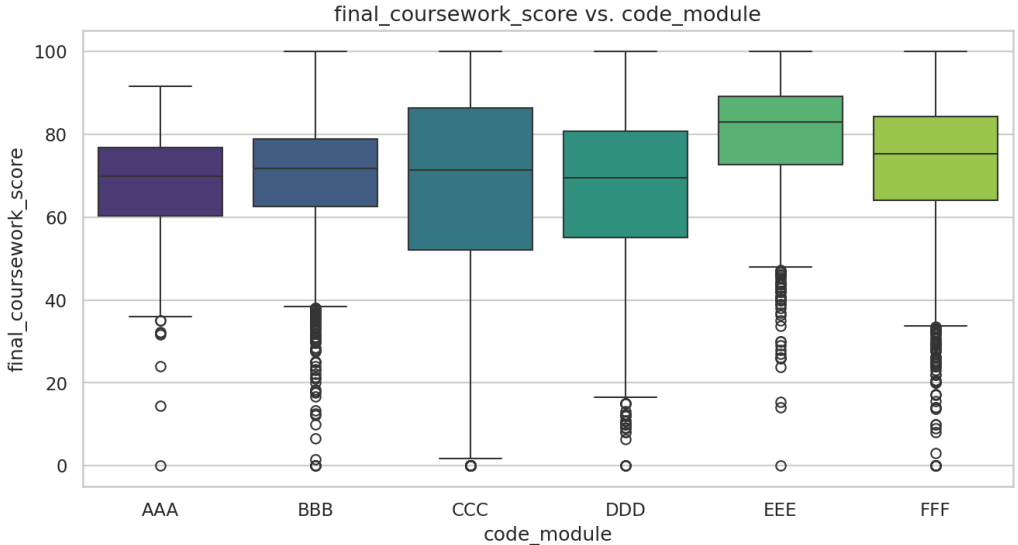
Atribut categorial -> Clasificare: **highest_education** vs **final_result** a fost analizata folosind Chi-square care este o metoda statistica de analiza a datelor categoriale cu scopul determinarii diferentei dintre frecventa observata vs frecventa asteptata. Rezultatul (Chi-square < 0.001) ne arata ca cele doua sunt dependente, studentii cu studii postuniversitare avand o rata mai mare de distinctie.



Atribut numeric -> Regresie: **mean_score_early** este puternic corelat cu **final_coursework_score** (0.82 din corelatia Perason). Studentii care completeaza evaluarile mai devreme tind sa aiba un rezultat mai bun la finalul cursului, fiind un bun predictor pentru regresie.



Atribut categorial -> Regresie: **code_module** infuenteaza notele, deoarece acestea au mediane diferite, sugerand dificultati variate.



3. Preprocesarea datelor

Pentru reproductibilitatea datelor am selectat un seed standard 42, destul de popular pentru task-uri de ML. De asemenea, pentru gestionarea usoara a seturilor de date pe care rulez, am integrat flag-ul “KAGGLE_MODE”, care pe False foloseste setul de test de pe Moodle (split 80/20 pe train), iar cand este pe True foloseste setul de test de pe Kaggle (train full de pe Moodle, test de pe Moodle ca validate). Rezultatele rularii modelelor sunt salvate in formatul descris pe Kaggle.

3.1. Imputarea valorilor lipsa

Bar plot-ul de mai devreme cu valorile lipsa ne arata foarte bine cam spre ce atribute trebuie sa ne orientam. Astfel, cele mai mari lipsuri sunt la **date_unregistration** (81%) pe care am hotarat sa il inlocuiesc cu **module_presentation_length,** deoarece un student care nu s-a retras a ramas pana la finalul modulului. Atributele de forma **clicks_*** le-am inlocuit cu 0, deoarece arata ca studentul nu a participat la acea activitate. **Imd_band,** dupa cum am specificat si mai sus, reprezinta indicele de calitate a internetului unui student si a fost inlocuit cu cea mai frecventa valoare in incercarea minimizarii bias-ului. **n_assessments_early, mean_score_early, weighted_mean_score_early, submission_rate_early** reprezinta performanta studentului, astfel au fost inlocuite cu mediana pentru reducerea outlierilor. **date_registration** nu cred ca face o dfierenta asa de mare, insa valorile lipsa am decis sa le imputez tot cu mediana.

De mentionat este ca statisticile de cea mai frecventa valoare si mediana au fost calculate exclusiv pe train, apoi aplicate pe val/test pentru a evita data leakage.

3.2. Tratarea outlierilor

Dupa EDA am selectat 10 atribute numerice care au mai multi outlieri vizibili. Metoda folosita este IQR x 1.5 (asa cum este specificat in enunt), valorile fiind capped la [Q1 – 1.5*IQR, Q3 + 1.5*IQR]. Fata de pasul anterior, limitele sunt calculate pe setul de train si aplicate doar pe cel de validare, nu si pe test pentru mentinerea unui scenariu cat mai real chiar si pe setul de test de pe Moodle.

3.3. Encodare atributelor

Conform recomandarii din enunt, am realizat encodarea atributelor categoriale. Pentru obiectivul **final_result** am folosit LabelEncoder, avand doar 4 clase distincte. Pentru

atributele nominale folosim One-Hot encoding, deoarece elimina presupunerea falsa a unei ordini numerice. Nu in ultimul rand, pentru atributele ordinale folosim Ordinal encoding care inlocuieste valorile categorice cu valori numerice.

3.4. Standardizarea datelor

Nimic special, doar StandardScaler din biblioteca scikit-learn pentru a ajuta in special modelele de Regresie Liniara/Logistica.

3.5. Pregatirea setului de test

Aici aplic imputarea si encodarea pe setul de test si pregatesc pentru CatBoost un format separat cu atribute categoriale ca string, fara encodare, fara standardizare, deoarece acesta le gestioneaza nativ.

4. Clasificare

4.1. Baseline: Arbore de decizie

Pentru acest model am facut o analiza detaliata pe 3 hyperparametrii:

- **max_depth**: specifica cat poate creste arborele; o valoare prea mica duce la underfit, iar o valoare prea mare duce la overfit, asa ca am ales mai multe valori, iar rezultatele s-au stabilizat undeva la max_depth = 10
- **min_samples_leaf**: numarul minim de exemple care trebuie sa ajunga intr-o frunza; o valoare mica duce la overfit, iar o valoare mare duce la underfit, iar rezultatele s-au stabilizat undeva la min_samples_leaf = 50
- **criteriul**: formula prin care arborele alege la fiecare nod ce atribut sa foloseasca pentru split; in general entropia ar trebui sa penalizeze mai mult impuritatile, insa in acest caz am avut rezultate mai bune cu Gini

4.2. Modele avansate: Random Forest, Logistic Regression si CatBoost

Random Forest reduce varianta arborelui de decizie prin medierea mai multor arbori. Pentru acest model am ales o configuratie mai stabila pentru hyperparametrii, fara fine tune special. **n_estimators** specifica numarul de arbori din padure, astfel am ales o valoare nu prea mica, dar nici prea mare care sa nu aduca un castig considerabil. Spre deosebire de un singur arbore de decizie, Random Forest permite o adancime mai mare a arborilor (am ales **max_depth**=20) fara risc de overfit, deoarece fiecare arbore vede date diferite, iar greselile nu se coreleaza. **min_sample_leaf**=5 de asemenea este un avantaj al

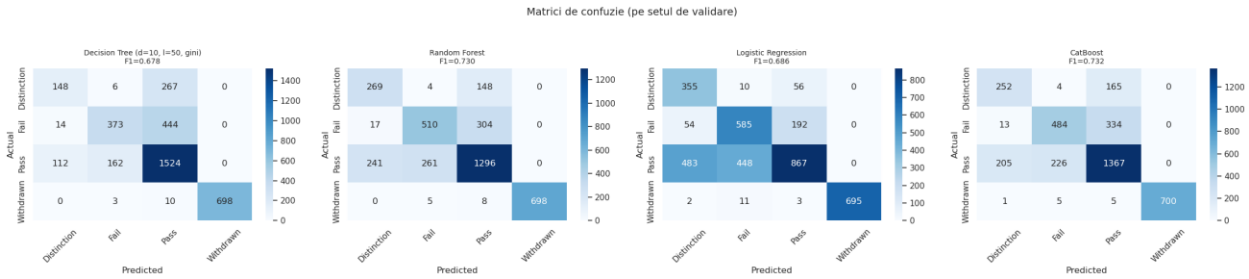
medierii Random Forest, fiecare frunza putand sa surprinda pattern-uri mai fine. **class_weight** a fost setat pe balanced pentru a penaliza greselile pe clasele mai rare (Distinction, Withdrawn) mai mult decat pe cele frecvente. Fara acest parametru, modelul ar ignora aproape complet clasa Distinction.

Am ales sa includ modelul **Logistic Regression** doar pentru prezentarea limitarilor sale in acest context (explicatie la urmatorul punct). Din acest motiv nu am insistat prea tare pe hyperparametrii, asteptandu-ma sa nu aiba rezultate bune. **max_iter**=1000 este necesar deoarece exista multe feature-uri categoriale encodate, insa cresterea valorii nu a adus rezultate mai bune. **class_weight** este exact ca la Random Forest. Din ce am inteles, hyperparametrul **solver** ar reprezenta algoritmul matematic de optimizare folosit de model, iar pentru probleme multiclasa (ca aceasta) recomandarea ar fi LBFGS, fiind rapid si stabil.

Nu in ultimul rand, dupa recomandarile din enuntul temei, am ales adaugarea CatBoost ce are la baza gradient boosting optimizat pentru date categoriale. Avand suport nativ si fiind un model ceva mai greu am ales rularea lui pe GPU (cu posibilitatea rularii pe CPU, daca nu exista un GPU compatibil). Aici m-am jucat mai mult cu hyperparametrii prin teste manuale. Numarul de iteratii este putin cam mare, fara imbunatatiri care sa fie semnificative, insa pe acest model m-am bazat cel mai mult (undeva la 500 de iteratii erau de ajuns, dar cu 2000 am mai scos putin din model). **Depth**=8 este undeva la limita superioara spre overfit, asadar nu am vrut sa cresc mai mult aceasta valoare (fata de RF aici arborii adanci overfit-eaza mai repede). **Learning_rate**=0.015 spune ca se fac pasi marunti, mai precisi spre predictie, astfel potrivindu-se cu numarul mai mare de iteratii. Combinatia **random_strength** si **bagging_temperature** fac modelul mai robust la overfitting prin introducerea factorului aleatoriu in fiecare split de feature-uri in faza constructiei arborilor (astfel previne modelul sa aleaga mereu acelasi split de feature-uri). Aici am observat un avantaj in folosirea clasei **SqrtBalanced** care este mai blanda cu ponderile pe clase.

4.3. Comparatia si analiza rezultatelor

Model	Accuracy	Precision_macro	Recall_macro	F1_macro
Decision Tree (d=10, l=50, gini)	0.729300	0.726200	0.657400	0.678300
Random Forest	0.737300	0.725600	0.738800	0.730200
Logistic Regression	0.665200	0.681900	0.751700	0.686000
CatBoost	**0.745300**	**0.734700**	**0.731500**	**0.731700**



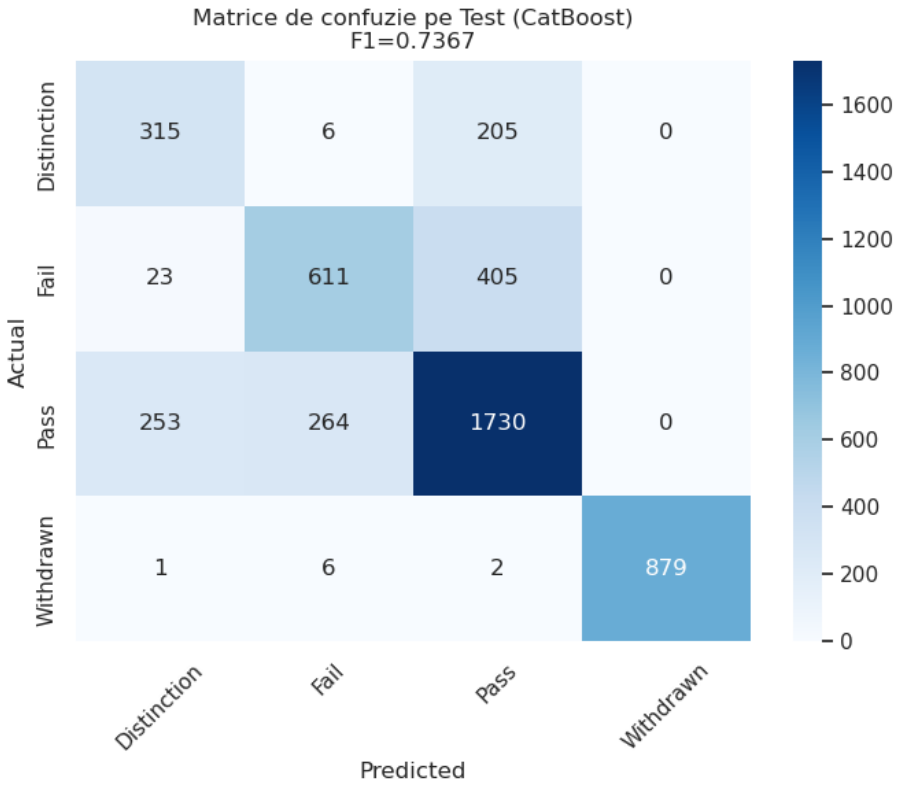
Rezultatele pe setul de validare ne arata cel mai bun model ca fiind CatBoost (cine ar fi crezut). Desi diferenta fata de Random Forest este destul de mica, CatBoost ofera un plus pe rezultate datorita gestionarii native a atributelor categoriale (nu are nevoie de encodare), algoritmul ordered boosting care reduce overfit-ul si capacitatea de a observa corelatii mai complexe intre atribute.

Logistic Regression arata un recall mult mai bun, insa cea mai slaba acuratete. Cam acesta este trade-off-ul modelelor liniare cu class_weight=balanced, deoarece incearca sa prinda toate clasele rare, dar fac greseli pe cele mai frecvente (se poate vedea asta in matricea de confuzie). Evident exista metode de imbunatatire a acestui model pentru clase non-liniare: termeni polinomiali, regularizare L1/L2 etc, insa consider ca depaseste scopul temei.

Din analiza matricelor de confuzie se observa cea mai mare eroare fiind intre clasele Fail si Pass, multi studenti pica desi au fost prezisi ca promovand. Aceasta nu este o eroare de model, ci probabil una de feature engineering (la care intr-adevar nu am lucrat foarte mult).

Comparatia de mai sus intre modele a fost realizata pe setul de validare. Rezultatele pe setul de test de pe Moodle sunt urmatoarele:

```
Evaluare clasificare pe setul de test (CatBoost)
  Accuracy:        0.7521
  Precision macro: 0.7399
  Recall macro:    0.7367
  F1 macro:        0.7367
```



Pe setul de test s-a folosit doar CatBoost si se observa o crestere usoara a metricelor fata de validare, ceea ce inseamna ca nu am overfit, iar generalizarea este buna.

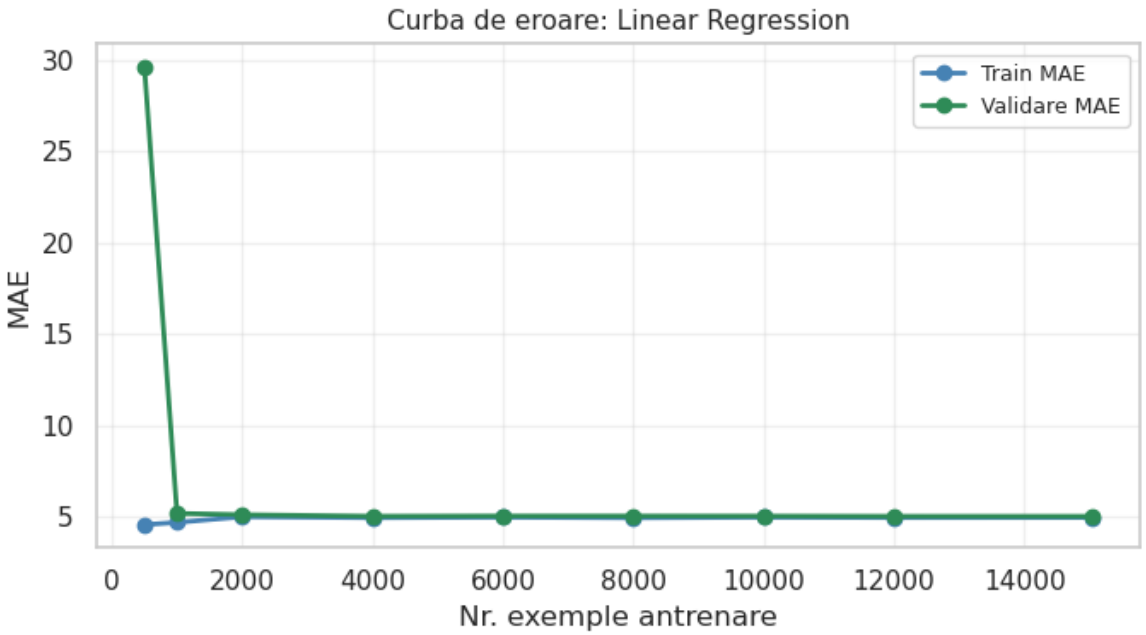
5. Regresie

Inainte de a incepe, am ales R2 ca metrica de comparatie pentru alegerea hyperparametrilor si a modelului cel mai bun, deoarece este normalizat si este mai comparabil intre dataset-uri (nu am nevoie sa stiu exact scala target-ului de exemplu, ci este o metrica care arata ca modelul este mai bun in proportie de X% decat media).

De asemenea, pentru fiecare model am antrenat pe subseturi crescatoare de date (500, 1000, 2000 etc) si am masurat curba de eroare pe train si val.

5.1. Baseline: Linear Regression + regularizare L1/L2

Initial am rulat fara regularizare. Modelul a incercat sa minimizeze MSE fara nicio constrangere pe coeficienti.



Pe grafic se observa curbele ce converg rapid cu un gap constant. Asta se intampla deoarece pe acest task, regresia liniara este depasita structural, adica exista relatii non-liniare precum cea dintre engagement si scorul final (un student cu 0 click-uri poate totusi promova daca prezinta temele) pe care nu le poate gestiona. De asemenea, este destul de sensibil la outlierii din setul de test.

Pe experimente cu diferite valori pentru regularizarile L1 si L2 nu s-au observat imbunatatiri, ba chiar degradari de performanta.

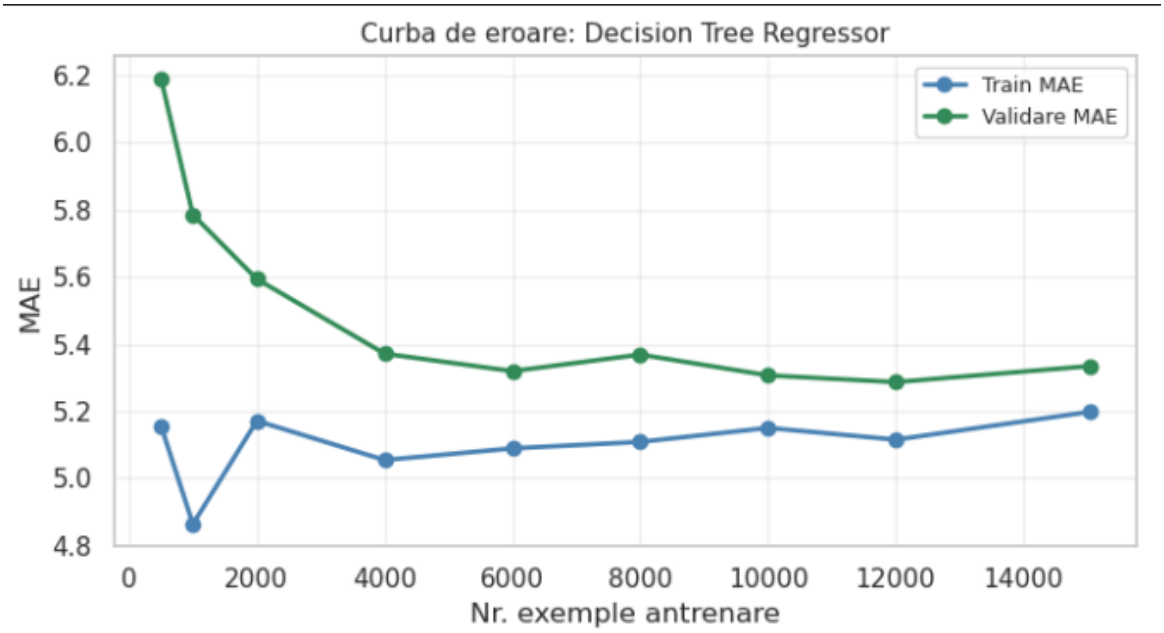
	Model	MAE_val	MSE_val	RMSE_val	R2_val	MAE_train	MSE_train	RMSE_train	R2_train
0	**Ridge a=0.01**	**5.032800**	**50.063000**	**7.075500**	**0.824800**	**4.974200**	**49.489300**	**7.034900**	**0.825300**
1	Ridge a=0.1	5.032800	50.063200	7.075500	0.824800	4.974200	49.489300	7.034900	0.825300
2	Ridge a=1.0	5.033000	50.064700	7.075600	0.824800	4.974300	49.489300	7.034900	0.825300
3	Ridge a=10.0	5.034100	50.080300	7.076700	0.824700	4.975000	49.490200	7.034900	0.825300
4	Ridge a=100.0	5.047600	50.261600	7.089500	0.824100	4.985600	49.534200	7.038100	0.825200

	Model	MAE_val	MSE_val	RMSE_val	R2_val	MAE_train	MSE_train	RMSE_train	R2_train
0	**Lasso a=0.01**	**5.024900**	**50.000000**	**7.071100**	**0.825000**	**4.971600**	**49.512700**	**7.036500**	**0.825200**
1	Lasso a=0.1	5.029200	50.283800	7.091100	0.824000	4.996700	50.132100	7.080400	0.823000
2	Lasso a=1.0	5.396300	55.100900	7.423000	0.807200	5.369400	55.009900	7.416900	0.805800
3	Lasso a=10.0	9.886600	156.802600	12.522100	0.451200	9.854700	155.325800	12.463000	0.451700

Rezultatele din test arata ca seturile de date pentru acest task nu sufera de o multicoliniaritate mai serioasa si ca feature-urile sunt complementare, nu redundante. Mentionez ca un experiment facut in etapa de preprocesare a datelor a fost eliminarea atributelor redundante, insa atat pentru clasificare, cat si pentru regresie am obtinut rezultate semnificativ mai proaste. Astfel, regularizarea liniara nu pare sa fie solutia potrivita aici, ci un avantaj l-ar considera modelele non-liniare.

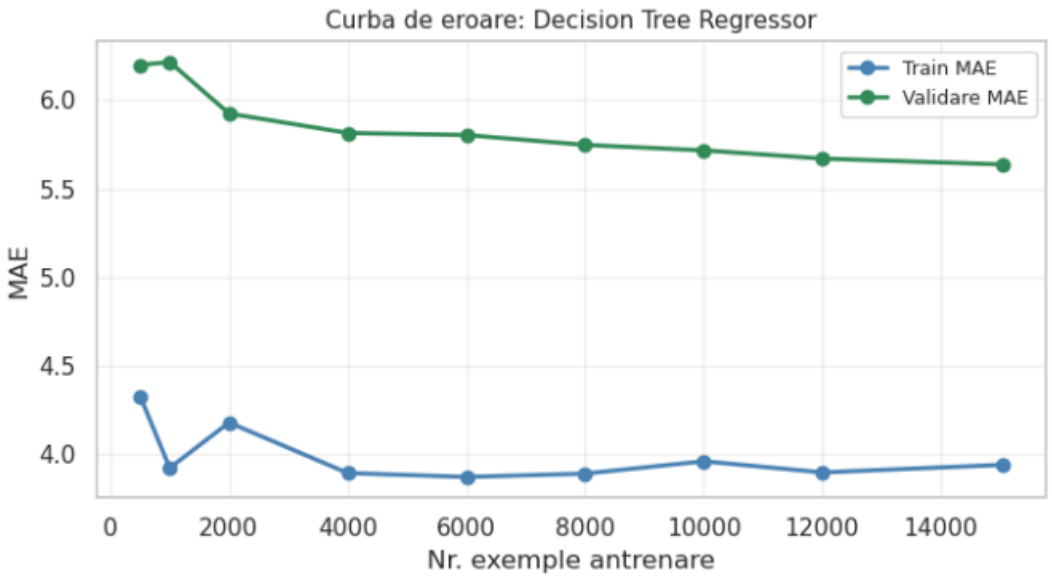
5.2. Modele avansate: Decision Tree Regressor, Random Forest si CatBoost

Astfel, ca baseline pentru modelel non-liniare am ales **Decision Tree Regressor**. Desi nu am pus prea mult accent pe fine-tuning-ul hyperparametrilor, as vrea sa prezint doua cazuri indentificate pentru acest model.



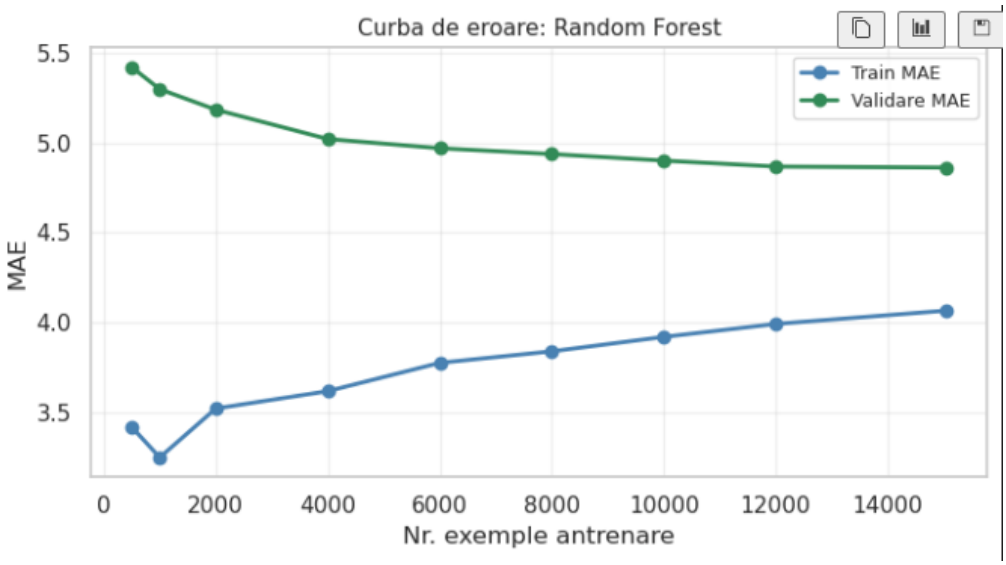
Primul dintre ele este si cel cu care am ramas pana la final, datorita rezultatelor mai bune. Folosind un **max_depth**=5 in comparatie cu unul mai mare, curba de eroare se stabilizeaza mult mai rapid, iar MAE scade putin, deoarece la regresie un arbore cu cat este mai adanc, cu atat capteaza mai mult zgomot. **min_samples_leaf=**20 ar trebui sa ofere destule exemple pe care sa se calculeze medii, ducand la o varibilitate mai redusa. Problema care intervine aici este in momentul in care rulam pe setul de test (la sectiunea de comparatie si analiza a rezultatelor) se poate observa vizual pe grafic cum modelul este

capabil sa faca doar 2^5 regiuni in spatiul de features, fiind nesuficient pentru captarea relatiilor non-liniare.



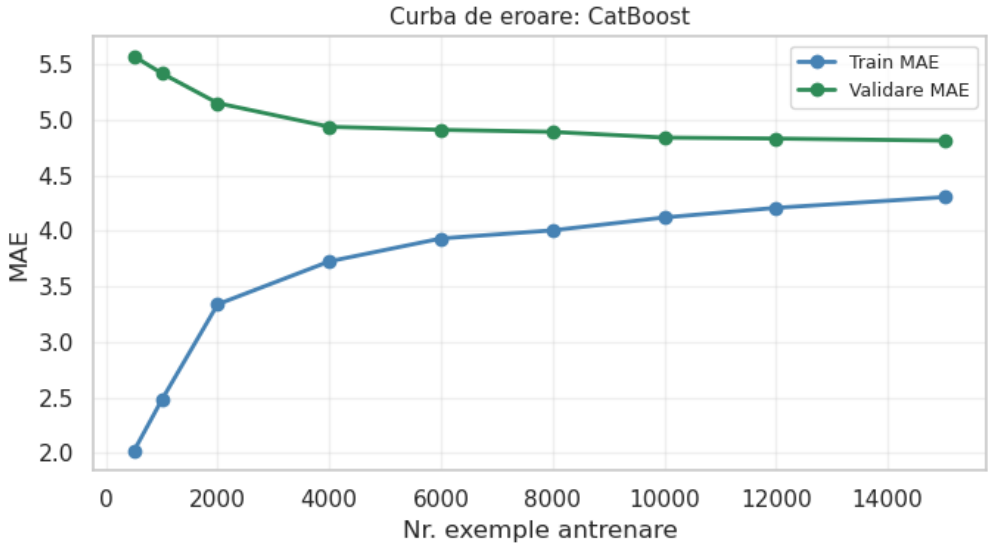
Al doilea caz identificat este cel in care am rulat cu **max_depth**=15 si **min_samples_leaf**=10. Aici am incercat sa fac opusul, si anume sa las modelul sa capteze mai multe relatii intre atribute impreuna cu frunze mai specifice, dar cu destula informatie. Din curba de eroare se observa ca modelul da overfit destul de mult pe setul de validare, avand rezultate mai proaste decat in primul caz. Modelul a creat niste reguli foarte specifice pe setul de antrenare, care s-au mulat pe noise si a dus la acest rezultat.

Aici intervine **Random Forest** care ar trebui sa reduca varianta de la DT semnificativ prin medierea mai multor arbori.



Aceasta curba de eroare este realizata pe configuratia urmatoare a hyperparametrilor: **n_estimators**=300, **max_depth**=10, **min_samples_leaf=**5. Pana sa ajung aici m-am jucat putin cu diferite combinatii intre max_depth si min_samples_leaf, deoarece initial existau diferente foarte mari intre train si val, sugerand un overfit. In acest sens, am redus max_depth pentru a exclude cazul in care un arbore s-ar adanci prea mult, iar frunzele ar ajunge sa aiba un singur rand de date, lucru care ar insemna ca modelul memoreaza fiecare caz in particular in loc sa gaseasca reguli care se aplica pe grupuri mai mari.

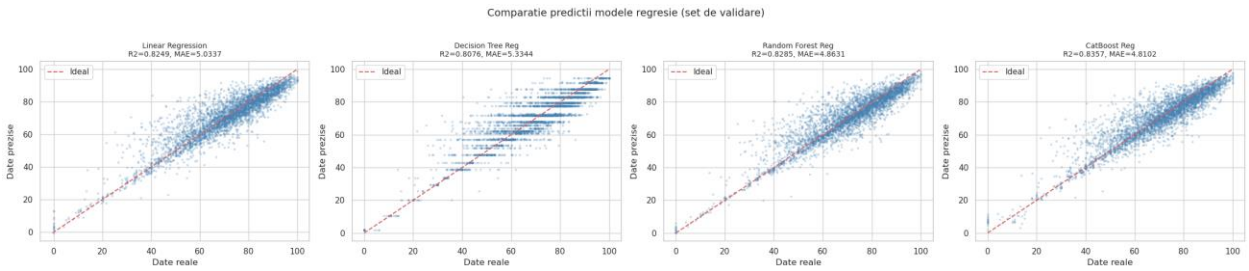
Ultimul model, pe care m-am bazat si pentru acest task, este **CatBoost**.



Deja se observa, in comparatie cu Random Forest, niste curbe de eroare mult mai apropiate si rezultate mai bune overall. Am decis pentru regresie sa reduc putin numarul de iteratii la 450 si sa maresc **learning_rate**=0.04 pentru o convergenta mai rapida din moment ce rulez modelul pe multe subseturi de date in scopul realizarii curbelor de eroare. Ce mai este diferit fata de modelul de clasificare, **min_data_in_leaf=**90 mai mare pentru a reduce overfit-ul. De asemenea, combinatia **random_strength**=0.35 si **bagging_temperature**=0.05 reduce aproape de tot acel sampling aleator de care am vorbit la clasificare, deoarece regresia este mult mai sensibila la zgomotul de sampling ce poate aduce variabilitate nedorita.

5.3. Comparatia si analiza rezultatelor

Model	MAE_val	MSE_val	RMSE_val	R2_val
Linear Regression	5.033700	50.044000	7.074200	0.824900
Decision Tree Reg	5.334400	54.985100	7.415200	0.807600
Random Forest Reg	4.863100	49.010200	7.000700	0.828500
CatBoost Reg	**4.810200**	**46.953100**	**6.852200**	**0.835700**



Rezultatele pe setul de validare ne arata cel mai bun model ca fiind tot CatBoost. Diferenta fata de Random Forest este putin mai sesizabila fata de taskul de clasificare, CatBoost ofererind un plus pe rezultate datorita gestionarii native a atributelor categoriale si gradient boosting-ul ajutand la corectia erorilor precedente (practic similar ca la clasificare).

Am fost mai mult surprins de Linear Regression care a reusit sa aduca R2 aproape de Random Forest. Acest lucru probabil se datoareaza faptului ca exista cateva feature-uri cu corelatie liniara foarte puternica la nota finala. Dupa cum am zis mai sus, modelul are limitari in intelegerea interactiunilor mai incrucisate intre atribute, fapt pentru care are un MAE/MSE mai slab.

Se poate observa graficul pentru DT avand o distributie neobisnuita a rezultatelor. Dupa cum am specificat la explicatia hyperparametrilor, am ramas pe configuratia cu max_depth=5. Aceasta limitare impune un arbore cu maximum 32 de frunze (rezultate posibile), adica fiecare student va primi unul din acele rezultate, astfel generandu-se pe axa OY niste nivele plate, distincte. Aceasta problema este remediata de Random Forest prin medierea a foarte multi arbori independenti.

6. Concluzii

Overall, tema a fost ok, insa cea mai mare dificultate intalnita a fost lipsa de explicatii/corectitudinea lor cu privire la seturile de date: in enunt erau specificate 3 csv-uri, noi aveam 2, ceva legat de labels in setul de test (care se referea la cel privat de care nu aveam idee atunci) si alte chestii de genul. De asemenea, explicatiile legate de aplicarea bonusului alaturi de platforma si valorile de baseline au venit foarte tarziu (apreciem totusi prelungirea deadline-ului) si nu consider ca au fost explicate cum trebuie (personal m-am consultat cu multi colegi observand interpretari diferite).

Trecand peste acest neajunsuri, putina experienta pe care am capatat-o din proiecte anterioare de ML m-a ajutat sa trec mai repede prin anumiti pasi si sa aplic niste configuratii folosite anterior. Totusi, tema m-a impins spre incercarea de modele noi si intelegere mai aprofundata a hyperparametrilor si a influentei lor. Tema e departe de a fi perfecta, fiind limitat atat de timp, cat si de scopul temei (nu limitari stricte in enunt, ci ideea generala a temei). Cu siguranta puteam sa imbunatatesc partea de preprocesare prin feature-engineering de exemplu si alegerea mai potrivita a hyperparametrilor folosind Optuna.